

## B 2.3.2 Selection Python



### B 2.3.2 Construct programs utilizing appropriate selection structures Python

Explore the power of if, else, and else if/elif statements. Learn how these structures, combined with Boolean and relational operators, allow programs to make decisions and execute different code blocks based on conditions. Dive into examples in Java and Python.

### Learning Objectives

- Understand the purpose and use of selection structures (if, else, else if/elif).
- Apply Boolean and relational operators to control program flow.
- Differentiate between if-else and multiple if statements.
- Implement selection structures within loops and nested loops.

### Introduction

Python implements selection structures using if, elif, and else, allowing programs to execute different blocks of code based on conditions.

### Basic if-else

The basic structure is similar to Java:

```
age = 20

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

This code checks if age is greater than or equal to 18 and prints a message accordingly.

## elif for Multiple Conditions

Python uses elif instead of else if:

```
grade = 85

if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("D")
```

This assigns a letter grade based on the numeric grade. In each line, it checks the condition. If any of the conditions are true the subsequent checks are skipped. If all the checks (if and elif) are completed and none are true the else statement outputs the grade "D".

## Boolean and Relational Operators

Python uses and, or, and not as Boolean operators, and relational operators (<, >, <=, >=, ==, !=) are used to construct complex conditions:

```
age = 25
is_employed = True

if age > 20 and is_employed:
    print("Eligible for loan.")
```

This code checks if the person is older than 20 **and** employed. Only when both conditions are true will the message "Eligible for loan" be printed.

# Nested Selection

You can nest selection structures in Python:

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("You can drive.")
    else:
        print("Get your license first.")
else:
    print("You are too young to drive.")
```

This checks age and license status. The first if statement checks whether the age is greater than or equal to 18. Only if that is true will the checks for the license be done. If age is not greater than or equal to 18 the the alternative message is printed.

## When to use nested if statements or boolean operators

Nested if statements and boolean operators (and, or, not) both provide ways to create more complex conditional logic in your code. Here's how they relate:

### Nested if Statements:

Create a hierarchy of conditions.

One if statement is nested inside another.

The inner if statement is only evaluated if the outer if statement's condition is true.

Useful for situations where you need to check multiple conditions in a specific order. For example as seen above:

```
age = 20
has_license = True

if age >= 18:
    if has_license:
        print("You can drive.")
    else:
        print("You need a license to drive.")
else:
    print("You are too young to drive.")
```

## Boolean Operators:

Combine multiple conditions into a single expression.  
Use operators like and, or, and not to create complex logic.  
All conditions within the expression are evaluated.

```
if age >= 18 and has_license:
    print("You can drive.")
else:
    print("You cannot drive.")
```

## Similarities:

Both allow you to express more complex conditions than a simple if statement.  
Both control the flow of your program based on whether conditions are met.

## Differences:

Nested-if statements create a hierarchical structure where conditions are evaluated in a specific order.  
Boolean operators combine conditions into a single expression where all conditions are evaluated.

## When to use which:

**Nested if:** Use when you need to check conditions in a specific order, and subsequent conditions depend on the outcome of previous ones.

**Boolean operators:** Use when you need to combine multiple conditions into a single expression and all conditions need to be evaluated.

In some cases, you can achieve the same logic using either nested if statements or boolean operators. However, choosing the right approach can make your code more readable and easier to understand.

## Key Questions - Python

How do if, elif, and else work in Python?

These keywords form the core of conditional statements in Python.

**if:** An if statement evaluates a boolean expression. If the expression is True, the code block indented under the if statement is executed.

**elif:** Short for "else if". If the preceding if condition is False, elif provides another boolean expression to evaluate. If the elif condition is True, its code block executes. You can have multiple elif blocks.

**else:** An optional final block. If none of the preceding if or elif conditions are True, the code within the else block executes.

Explain the difference between and, or, and not operators.

These are Python's logical operators for combining boolean expressions.

- **and:** Returns True only if both expressions on either side are True.
- **or:** Returns True if at least one of the expressions on either side is True.
- **not:** Returns the opposite boolean value of the expression it precedes (e.g., not True is False).

When is it appropriate to use an if-elif-else chain instead of multiple if statements?

**if-elif-else:** Use when you have a series of mutually exclusive conditions, where only one block of code should be executed. Once a condition is met, the rest of the chain is skipped. This is efficient and prevents unintended consequences.

**Multiple if statements:** Use when each condition needs to be evaluated independently, and it's possible that multiple code blocks might need to execute.

How can selection structures be used within loops to affect which actions are repeated?

Selection structures within loops allow you to conditionally execute code during each iteration. This gives you the flexibility to:

- **Skip certain iterations:** Use if to decide if a block of code should be executed in a specific iteration.

- **Modify loop behaviour:** Change the values of variables used in the loop condition to influence how many times the loop runs.
- **Perform different actions:** Use if-elif-else to choose different code paths within the loop based on specific conditions.

Provide a real-world scenario where nested if statements would be necessary in a Python program.

Nested if statements are essential when dealing with hierarchical decisions or multiple layers of conditions.

- **User authentication:** Check if a user exists, then if their password matches.
- **Data validation:** Check if data is of the correct type, then if it falls within an acceptable range.
- **Game AI:** Make decisions for a game character based on its surroundings and the player's actions.
- **Chatbots:** Guide conversation flow based on user input and context.
- **Recommendation systems:** Filter products or content based on user preferences and history.

---

*All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.*

---

 **Report a problem**